

Chapter 8 Summary — Safe Exploitation in Imperfect-Information Games

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

July 2026

Contents

8 Chapter 8 — Safe Exploitation in Imperfect-Information Games	1
8.1 Why Safe Exploitation — the other half of the dial	2
8.2 Exploitation as Constrained Optimization	3
8.3 What “Safe” Means — three definitions, in increasing realism	4
8.4 Sequence-Form LP and Constraint Generation	7
8.5 Restricted Nash Response and the Bang-Bang Switch	7
8.6 Best Equilibrium on Kuhn — safe, but only a little more profitable (the core result) . . .	10
8.7 Real-Time Safety — subgame gadgets (SES)	12
8.8 At Scale — Kuhn works; Leduc breaks globally, holds locally	13
8.9 Connections and Forward Pointers	17

8 Chapter 8 — Safe Exploitation in Imperfect-Information Games

This is a ground-up chapter on *safe* opponent exploitation: the problem, the mathematics, the family of methods, and a set of controlled experiments run on two small poker games. All experimental numbers reported here were **measured** on reproducible runs of the two testbeds (Kuhn Poker and Leduc Hold'em) and are bounded, wherever possible, by *exact* analytical references rather than simulated ones. Where a run contradicted what theory led me to expect, I say so and reconcile it — those gaps are the most instructive parts of the chapter.

Where this sits in the thesis. Chapter 7 built a **sensor**: a model that watches how a specific opponent plays and estimates their strategy. It also exposed the danger of acting on that sensor naively — a confident-but-underfit model, best-responded to, can open a hole in your own play (on Leduc it actually *lost* to an unexploitable Nash opponent). Chapter 8 builds the **actuator**: the mechanism that turns a model into profit **without becoming exploitable**. That actuator is the second half of the thesis's Behavioral Adaptation Framework (Contribution #1), and pinning down *exactly where its guarantees depend on the two-player zero-sum assumption* is the launch point for the multi-agent extension (Contribution #2).

8.1 Why Safe Exploitation — the other half of the dial

A **Nash equilibrium** strategy is built never to lose in the long run. In a two-player zero-sum game that guarantee is exact: the strategy has a fixed *value* v^* , and no opponent, however clever, can push you below it. Opponent modeling (Chapter 7) buys you the ability to *deviate* from that safe strategy to punish a specific opponent’s mistakes. The obvious way to cash a model in is to compute the **best response** to it and play that. The trouble is that a best response is usually **wildly exploitable itself**: to punish “you always fold to a bet” it stops bluffing certain hands entirely, and a smarter opponent — or the same opponent, done pretending — walks straight through the hole it opened.

Picture a **dial**. Turn it all the way to *safety* and you play Nash: unbeatable, but you never punish a weak opponent. Turn it all the way to *exploitation* and you best-respond hard to your current read: maximum profit **if** the read is right and the opponent stays put, but a gaping hole if you are wrong, your sample was small, or you were being *sandbagged* (deliberately fed a weak style to bait a big deviation). **Safe exploitation** bolts a *governor* onto that dial: lean toward exploitation as far as you like, but never past the point where a worst-case adversary could drag you below your safe baseline.

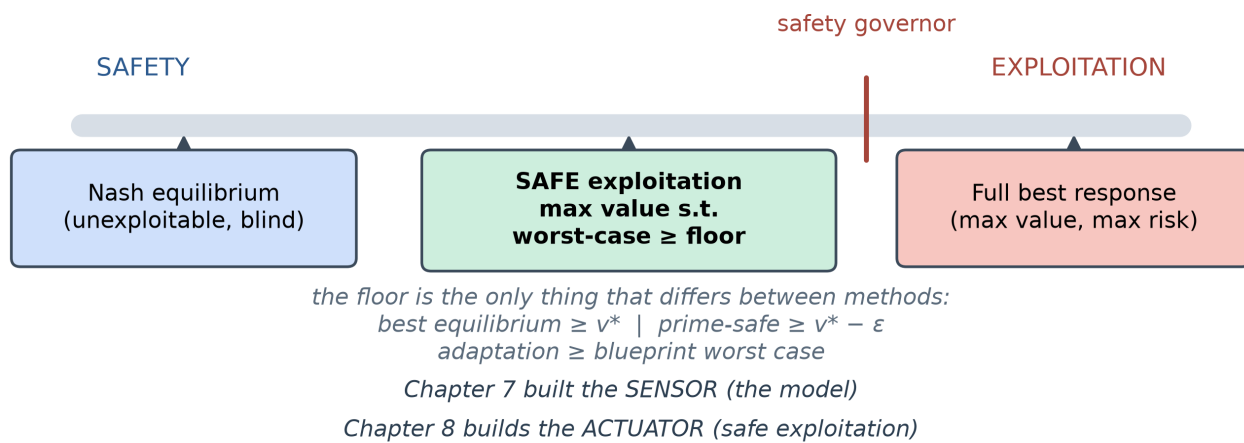


Figure 1: The safety-exploitation dial with a governor. Pure Nash is unexploitable but blind; a full best response extracts the most value but is maximally risky. Safe exploitation operates in between, capped by a floor on the worst-case value. Chapter 7 built the sensor (the model); Chapter 8 builds the actuator (safe exploitation). The only thing that differs between methods is where the floor sits.

The reason this is not paranoia is a number from Chapter 7 and again from this chapter: the full best response to the tight “Rock” style on Kuhn earns **+0.167 per hand**, but its **worst-case value is −0.5** — an adversary who best-responds back can take half a chip a hand off it. Nash’s worst-case, by contrast, is the game value itself. Safe exploitation is the discipline of taking as much of that upside as the opponent’s mistakes allow while refusing the −0.5 downside.

What “safe” means, informally. Your *expected value over the match* must never fall below a chosen floor, **no matter what the opponent does**. It is a statement about the worst case, not about any single

hand — you will still lose hands; the guarantee is on the long-run adversarial average. The whole design question of the chapter is *which floor*, and how to compute a strategy that respects it while extracting as much as possible above it.¹

8.2 Exploitation as Constrained Optimization

The single most useful idea in this chapter is that **every safe-exploitation method is the same optimization problem** with one part swapped out. In words:

maximize the hero’s expected value against the opponent model, **subject to** a safety floor on the hero’s worst-case value.

To make that a *computable* program, represent the hero’s strategy in **sequence form**, introduced in Section 7.5 for the opponent’s strategy: the **realization plan** x assigns a weight to each of the hero’s action *sequences* (root-to-her chains of their own choices), subject to the linear **treeplex** constraints (the empty sequence has weight 1; at each information set the children’s weights sum to the parent’s; all weights are non-negative). The reason to pay this change-of-variables tax is decisive: against a **fixed** opponent policy, the hero’s expected value is **linear** in x ,

$$\text{EV}(x) = \sum_{z \in Z} [\pi_c(z) \cdot \pi_{-h}(z) \cdot u_h(z)] x_{\text{seq}(z)} = c \cdot x,$$

where Z is the set of terminal states, $\pi_c(z)$ the probability of the chance moves, $\pi_{-h}(z)$ the probability of the opponent’s moves and $u_h(z)$ the hero’s payoff. This holds because the hero’s own action-probability product along any line *is* x at that line’s sequence. So “maximize EV against the model” is a **linear objective** $c \cdot x$, the treeplex is a set of **linear constraints**, and the whole method is a **linear program**. A full best response is simply $\max_x c \cdot x$ over the treeplex — which lets us cross-check the LP against Chapter 7’s exact best-response code (they agree to 10^{-6}).

The safety floor is a constraint on the **worst-case** value — “for *every* opponent σ' , $\text{EV}(x, \sigma') \geq \text{floor}$.” That inner “for every opponent” is itself a best-response (a minimax), so we do not write it out as one giant bilinear program; we solve it by **constraint generation** (a double-oracle / cutting-plane loop): solve the LP, read the hero policy, call an **exact best response as the worst-case oracle**, and if the worst case is below the floor, add the single linear cut it implies and re-solve. This is finite (there are finitely many pure best responses), transparent, and debuggable.

¹Ganzfried, S. & Sandholm, T. (2015). “Safe Opponent Exploitation.” *ACM Transactions on Economics and Computation* 3(2), Article 8. DOI 10.1145/2716322 — the paper that shows that, in a repeated two-player zero-sum game, deviating from equilibrium can be safe exactly when the opponent makes “gifts”, and gives algorithms that risk only what has already been won; the safety half of the dial and the anchor for Chapter 8.

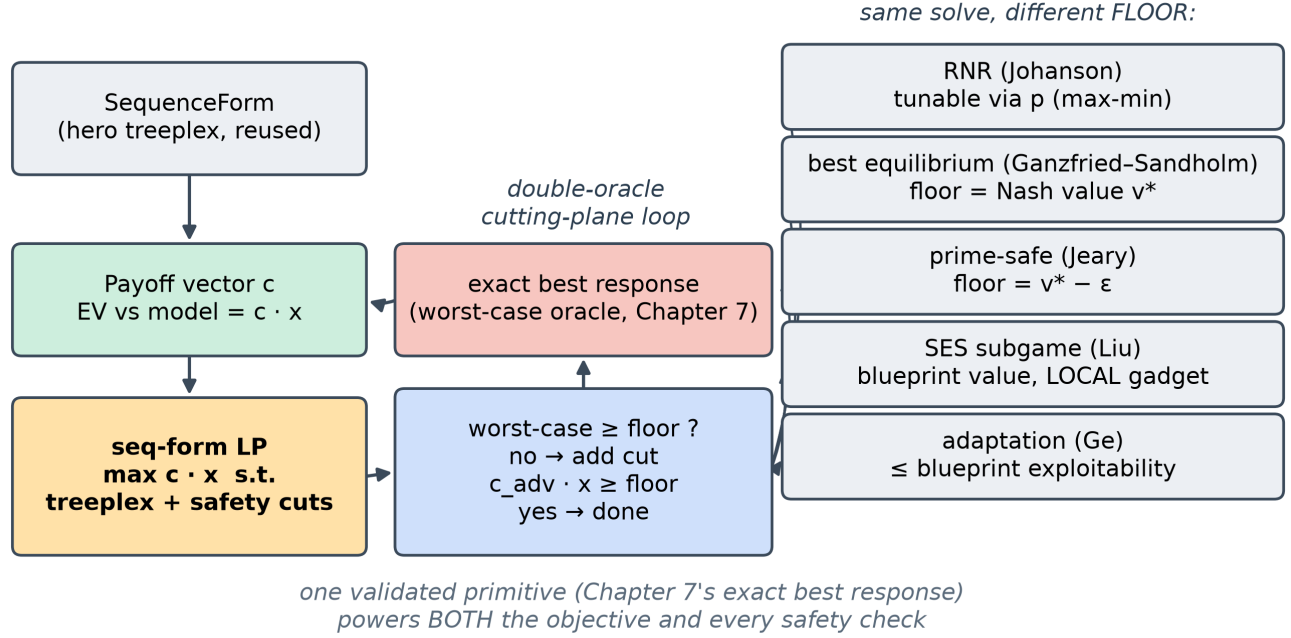


Figure 2: One LP engine, five safety floors. The sequence-form treeplex gives a linear objective (EV vs the model); a constraint-generation loop calls an exact best response as the worst-case oracle and adds a safety cut until the floor is met. RNR, best equilibrium (the Ganzfried–Sandholm baseline), prime-safe, SES and adaptation are the SAME solve with a different floor — and one validated primitive (Chapter 7’s best response) powers both the objective and every safety check.

The pay-off of seeing it this way is conceptual economy: **five methods, one engine**. They differ only in the floor, which is the subject of the next section. And because both the objective (the payoff vector) and the safety oracle reuse Chapter 7’s *validated* best-response code, the whole of Chapter 8 rests on one already-trusted primitive rather than a new pile of math.²

8.3 What “Safe” Means — three definitions, in increasing realism

“Safety” sounds absolute, but it is not one thing. Three definitions matter, and the line of research is essentially a *weakening* of the demand to make it achievable in practice.

(a) **Ganzfried–Sandholm safety (2015): never earn less than the game value.** Their definition is over the repeated game: at least v^* per hand in expectation, whatever the opponent does. Here it is imposed on each hand’s strategy, with an exact Nash equilibrium σ^* as baseline:

²Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. Ch. 3 (normal-form games); Ch. 4 (computing solution concepts of normal-form games; §4.1, linear programming for zero-sum games); Ch. 5 (extensive-form games; §5.2, imperfect-information games and the sequence form, the machinery underneath every LP in Chapter 8); Ch. 7 “Learning and Teaching”, the learning-in-repeated-games framing, including the tension that your actions both *exploit* and *teach* the opponent. Free: <http://www.masfoundations.org/download.html>

$$\text{EV}(x, \sigma') \geq v^* \quad \text{for all opponents } \sigma'.$$

This is the strongest and cleanest notion. It rests on the **minimax theorem**: in a two-player zero-sum game a Nash strategy *guarantees* the value v^* against any opponent. Only equilibrium strategies meet this per-hand floor, so the method picks the equilibrium strategy that does best against the model (Ganzfried and Sandholm’s “best equilibrium” baseline; below it is called **best equilibrium**, code id **ganzfried**). Their full algorithms go further: they deviate beyond equilibrium by risking only the *gifts* (the opponent’s mistakes) already banked in earlier hands. The catch is the premise: an exact Nash equilibrium, which is not computable in practice in any large game.

What the full algorithms would add. Their RWYWE (“risk what you’ve won in expectation”) solves the same LP each hand with the floor lowered to $v^* - k_t$, where k_t is the gift banked in expectation so far, so the strategy moves past equilibrium exactly as far as the opponent’s mistakes have paid for; BEFFE plays best equilibrium until the banked gifts cover full exploitation for the rest of the match. Both are provably safe over the repeated game and can exploit more than best equilibrium; implementing and testing them is future work (RWYWE is the natural two-player baseline for Contribution #2).

(b) Prime-safe / ε -safety (Jeary & Turrini 2023): correct for an imperfect baseline. Every real baseline is an **ε -equilibrium** (from abstraction and finite compute — the subject of Chapter 4), so it is itself exploitable by some amount ε . Anchoring to the exact v^* is then unjustified; prime-safe lowers the floor by exactly the baseline’s own exploitability,

$$\text{floor} = v^* - \varepsilon, \quad \varepsilon = \text{exploitability}(\text{baseline}) \geq 0,$$

i.e. “never earn less than the worst case of the strategy you were going to play anyway.” The ε must be **measured**, not assumed — a point the experiments take literally.

(c) Adaptation safety (Ge et al. 2024): be no more exploitable than your blueprint. The most practical notion. An exploiting strategy is *adaptation-safe* iff

$$\text{exploitability}(x) \leq \text{exploitability}(\text{blueprint}) \iff \text{worst-case}(x) \geq \text{worst-case}(\text{blueprint}).$$

Because the blueprint is already ε -exploitable, this is strictly weaker than Ganzfried and Sandholm’s absolute floor (weaker by exactly ε), and therefore *achievable* where strict safety is not. Its danger is the mirror image: if the blueprint is *terrible*, “no worse than the blueprint” is trivially satisfied by almost anything — so adaptation safety is only meaningful with a reasonable baseline (an open requirement I flag rather than resolve).

Table 1: Three safety notions: the floor each guarantees, what it requires, and where it is weak.

Safety notion	Informal floor	Needs	Weakness
Ganzfried–Sandholm (2015)	$\geq v^*$	an <i>exact</i> Nash baseline	an exact Nash equilibrium is not computable in practice in large games
Prime-safe (2023)	$\geq v^* - \varepsilon$	the baseline’s measured exploitability	you must measure ε honestly
Adaptation (2024)	$\leq$ blueprint exploitability	any blueprint	vacuous if the blueprint is bad

8.3.1 Where the two-player zero-sum assumption hides — the thesis attack point

Every floor above rests on one fact: **in a two-player zero-sum game, a Nash strategy secures the value v^* against any opponent** ($\min_{\sigma'} \text{EV}(\sigma^*, \sigma') = v^*$). That is what makes “deviate toward the model but never below the floor” a coherent, enforceable constraint, and it comes straight from the minimax theorem. In an $N > 2$ -player game this collapses: a Nash strategy does *not* guarantee a fixed value against arbitrary opponents (no single opponent’s payoff is the negative of yours any more — in a zero-sum game that holds only for the opponents’ payoffs taken together — so the opponents can act in concert against you, and different equilibria pay you differently), so there is no single v^* to anchor to. Equilibria in games with more than two players are “neither unique nor non-exploitable”.³ N -player safety notions do exist, but they are either very conservative (the maxmin value against a coordinated coalition⁴) or a target that cannot be secured in general (equal share, the fair share C/n , is provably not securable against opponents with different fixed strategies³). **That is the open problem for Contribution #2** — exploiting weak opponents in small N -player games with a bounded loss relative to a baseline strategy, including against colluding opponents. This chapter does not solve it; it makes the failure *precise*, which is exactly what a thesis chapter needs.⁵⁶⁷

³Ge, J., Wang, Y., Li, W. & Jin, C. (2025). “Securing Equal Share: A Principled Approach for Learning Multiplayer Symmetric Games.” *ICML*; arXiv:2406.04201.

⁴Celli, A. & Gatti, N. (2018). “Computational Results for Extensive-Form Adversarial Team Games.” *AAAI* 32(1). DOI 10.1609/aaai.v32i1.11462.

⁵Johanson, M., Zinkevich, M. & Bowling, M. (2007). “Computing Robust Counter-Strategies.” *NIPS 2007*, 721–728 — Restricted Nash Response, the tunable ancestor of the methods above.

⁶Jeary, L. & Turrini, P. (2023). “Safe Opponent Exploitation For Epsilon Equilibrium Strategies.” *arXiv:2307.12338*.

⁷Ge, Z., Xu, Z., Ding, T., Meng, L., An, B., Li, W. & Gao, Y. (2024). “Safe and Robust Subgame Exploitation in Imperfect Information Games.” *ICML*, PMLR 235, 15255–15270.

8.4 Sequence-Form LP and Constraint Generation

This section is the “how it is built.” The one primitive the whole chapter adds is a **HeroTreeplex** that (i) enumerates the hero’s sequences and treeplex constraints (reusing Chapter 7’s sequence-form code), (ii) builds the payoff vector c for a fixed opponent by one tree traversal, and (iii) solves the safety-constrained LP with SciPy’s HiGHS solver. Everything else is the constraint-generation loop, which is short enough to state in full:

```
c_model <- payoff vector of EV vs the opponent model      # linear objective
cuts    <- {}                                             # discovered safety cuts
repeat:
  x <- argmax c_model . x s.t. treeplex(x) and all cuts    # one LP solve
  pol <- behavioral strategy read out of x
  wc <- worst_case_value(pol)    # = -(opp. exact BR value): the oracle
  if wc >= floor - tol: return pol    # safe: done
  adv <- opponent's exact best response to pol
  add cut c(adv) . x >= floor - slack # valid lower bound on worst case
```

Each generated cut is the payoff vector against a *specific* adversary best response; it is a valid linear lower bound on the true worst case, so adding cuts monotonically tightens the relaxed safety constraint toward the real one. Reading a behavioral policy back out of a realization plan x is the ratio $\beta(I, a) = x_{Ia}/x_{\text{seq}(I)}$ (uniform where the parent weight is ~ 0).

The **only** things that change between the five methods are the **floor** and, for the subgame method (§ 8.7), a set of *pins* that hold the hero’s play outside a chosen subgame equal to the blueprint. Best equilibrium passes **floor** = **v***; prime-safe passes **floor** = **v*** - ϵ ; adaptation passes **floor** = **worst_case(blueprint)**; RNR reformulates the objective as a max-min in a parameter p (§ 8.5).

One practical wrinkle worth recording (it cost real debugging on the runs): an *approximate* Nash’s self-play value can sit a hair **above** the game’s true achievable max-min, so requiring the exact **wc** $\geq$ **floor** can make the LP eventually infeasible and, on an unlucky cut path, return an *unsafe* strategy. The fix is a small feasibility slack on the cuts (**floor** - **slack**, with **slack** $\approx 5 \cdot 10^{-4}$, kept independent of the convergence tolerance) so the true max-min strategy always stays feasible. This is the kind of numerical detail that never appears in the papers but decides whether the method actually returns a safe strategy.⁸

8.5 Restricted Nash Response and the Bang-Bang Switch

One of the earliest principled methods, **Restricted Nash Response** (Johanson 2007; close to the ϵ -safe strategies of McCracken and Bowling, 2004), is the conceptual ancestor of all the rest and introduces the *tunable knob*. Its idea: compute the hero’s equilibrium against a **p -restricted** opponent — one forced to

⁸The constraint-generation / double-oracle idea traces to McMahan, H. B., Gordon, G. J. & Blum, A. (2003). “Planning in the Presence of Cost Functions Controlled by an Adversary.” *ICML* — the general recipe for “optimize against a worst case you discover as you go.”

play the fixed model with probability p and free to play adversarially with probability $1 - p$. Sweeping p from 0 to 1 traces a path from Nash ($p = 0$) to full best response ($p = 1$). Because the free component best-responds to the hero, RNR is a **max-min**:

$$\text{RNR}(p) = \arg \max_x \left[p \cdot \text{EV}(x, \text{model}) + (1 - p) \cdot \min_{\sigma'} \text{EV}(x, \sigma') \right],$$

which we solve with the same cutting-plane machinery plus an auxiliary worst-case variable.

A flag worth keeping. The project’s own step notes originally described RNR as a *naive behavioral blend*, $(1 - p) \cdot \text{Nash} + p \cdot \text{BR}$. That is a fine intuition tool but it is **not** Johanson’s algorithm; we implemented *both* and labelled them, because on a small game they behave completely differently. The naive blend traces a smooth line (the grey line in the next figure): profit and exploitability rise together, near-proportionally, from Nash to full BR.

The measured surprise — canonical RNR is bang-bang. I predicted the canonical sweep would be a smooth, monotone frontier that dominates the blend everywhere. It is not. On Kuhn versus the Rock, canonical RNR returns the **same safe strategy** for all $p \in [0, 0.6]$ (EV -0.044 , exploitability ≈ 0) and then **jumps straight to the full best response** at $p \approx 0.7$ (EV $+0.167$, exploitability 0.444). Between the sampled values of p (steps of 0.1) there are no intermediate points.

Table 2: Restricted Nash Response across the mixing parameter: the canonical form against a naive blend.

p	canonical RNR — EV	canonical — exploitability	naive blend — EV	naive — exploitability
0.0	-0.044	~ 0.000	-0.047	0.000
0.3	-0.044	~ 0.000	$+0.017$	0.133
0.6	-0.044	~ 0.000	$+0.081$	0.266
0.7	$+0.167$	0.444	$+0.103$	0.311
1.0	$+0.167$	0.444	$+0.167$	0.444

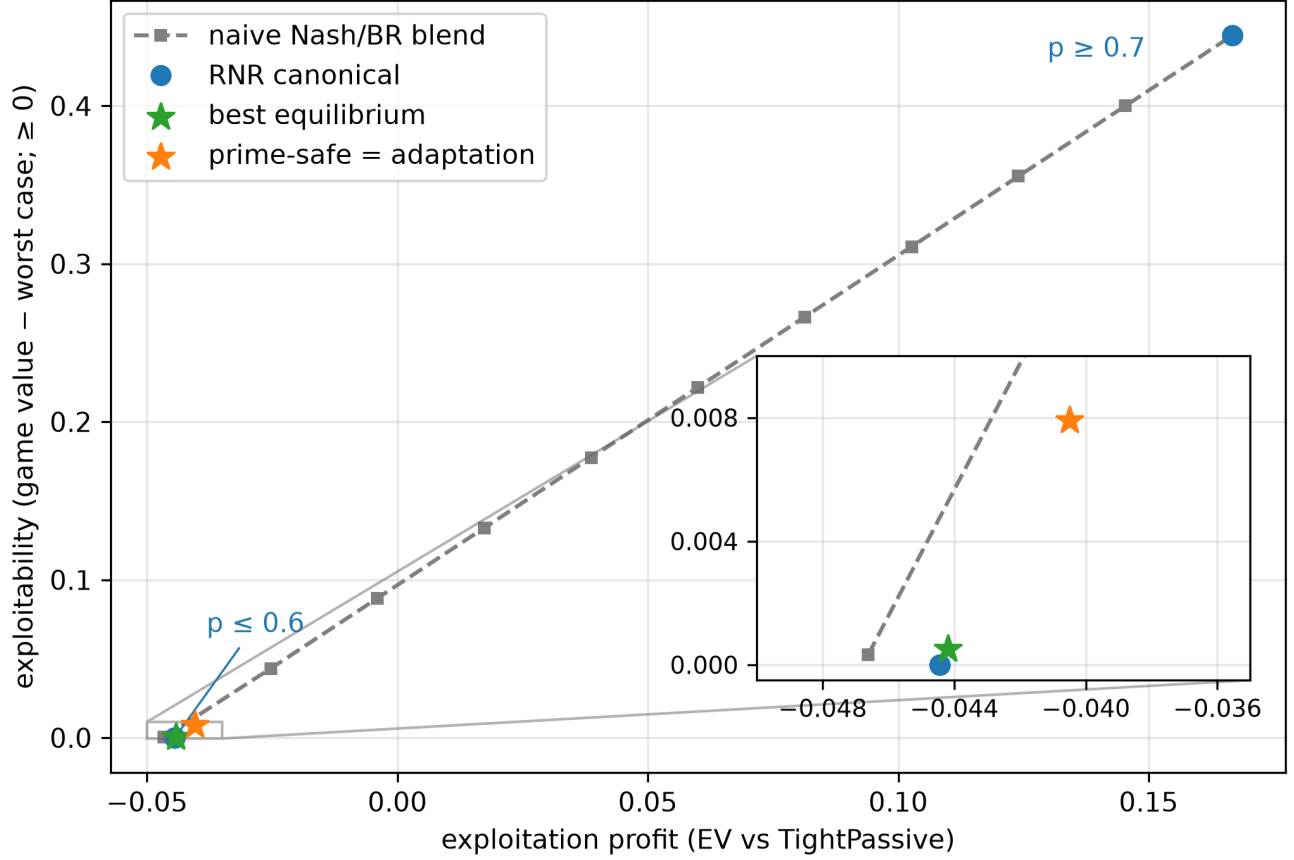


Figure 3: The exploitation-safety frontier on Kuhn with the LP operating points. Canonical RNR returns only two clusters of points (the safe corner for $p \leq 0.6$ and the full-BR corner for $p \geq 0.7$); the grey line is the dominated naive blend; the stars (best equilibrium, and prime-safe and adaptation, which coincide) sit at the efficient safe corner, enlarged in the inset.

Why (I checked before trusting the story). The RNR objective is **linear** in x over a **polytope**, so its optimum is a **vertex** and switches vertices only when p crosses a critical ratio. Kuhn’s strategy polytope is tiny — few vertices — so the transition is a single jump. Johanson’s smooth curve is a *large-game* phenomenon (many vertices), or comes from the *data-biased* variant that makes p per-information-set. The naive blend looks smooth only because it linearly interpolates two fixed strategies — and it is **dominated at the safe corner** (at exploitability ≈ 0 the LP methods reach EV -0.044 versus the blend’s -0.047). What is bang-bang is the map from p to a strategy, not the frontier: mixing the two RNR strategies (Johanson et al. define this mixture) reaches every point on the chord, because EV is linear in the mixture and the worst case is concave. With the two vertices found, the switch happens at $p \approx 0.68$; any further vertex would be optimal only for p between 0.6 and 0.7, which the sweep did not sample. Against the Rock the chord improves on the naive blend by only ≈ 0.002 , so here choosing *where* to deviate buys almost nothing over uniform scaling; Johanson et al. report strongly concave frontiers in a large abstracted hold’em game. What the small game does show is that p is not a dial: the strategy switches from the safe vertex to the BR vertex at a threshold that *moves with how exploitable the opponent*

is. That last point recurs in § 8.6: against a very exploitable opponent, even $p = 0.5$ is already past the jump.⁹

8.6 Best Equilibrium on Kuhn — safe, but only a little more profitable (the core result)

The central experiment solves each method against a *perfect* model of each opponent type and scores it on **profit** (EV vs that opponent) and **safety** (worst-case value), with the Nash floor at $v^* = -0.056$ (Kuhn’s known first-player value, $\approx -1/18$). All numbers are exact (full-tree), from the scale run.

Table 3: Kuhn: expected value and worst-case value for every method against every opponent (“best eq.” is the best-equilibrium baseline, code id **ganzfried**).

Opponent	metric	nash	full_br	rnr_0.5	best eq.	prime_safe	adaptation
Rock (TightPassive)	EV	−0.047	+0.167	−0.044	−0.044	−0.040	−0.040
	worst-case	−0.056	−0.500	−0.056	−0.056	−0.063	−0.063
Maniac (LooseAggr.)	EV	+0.118	+0.333	+0.278	+0.131	+0.151	+0.151
	worst-case	−0.056	−0.167	−0.111	−0.056	−0.063	−0.063
AlwaysBet	EV	+0.113	+0.333	+0.333	+0.115	+0.131	+0.131
	worst-case	−0.056	−0.167	−0.167	−0.056	−0.063	−0.063
AlwaysPass (most exploitable)	EV	+0.146	+0.975	+0.975	+0.222	+0.266	+0.266
	worst-case	−0.056	−0.500	−0.333	−0.056	−0.063	−0.063
Nash (control)	EV	−0.056	−0.055	−0.056	−0.056	−0.055	−0.055
	worst-case	−0.056	−0.333	−0.056	−0.056	−0.063	−0.063

(**ses_subgame** is not shown: on Kuhn the subgame is the whole game and the method coincides with adaptation. The sixth type in the data, **Thresholdish**, gives the same picture: best equilibrium +0.019 versus +0.015 for Nash.)

⁹Johanson, M. & Bowling, M. (2009). “Data Biased Robust Counter Strategies.” *AISTATS*, PMLR 5, 264–271 — makes the p knob depend on how much data supports each part of the strategy, which is what smooths the frontier in practice.

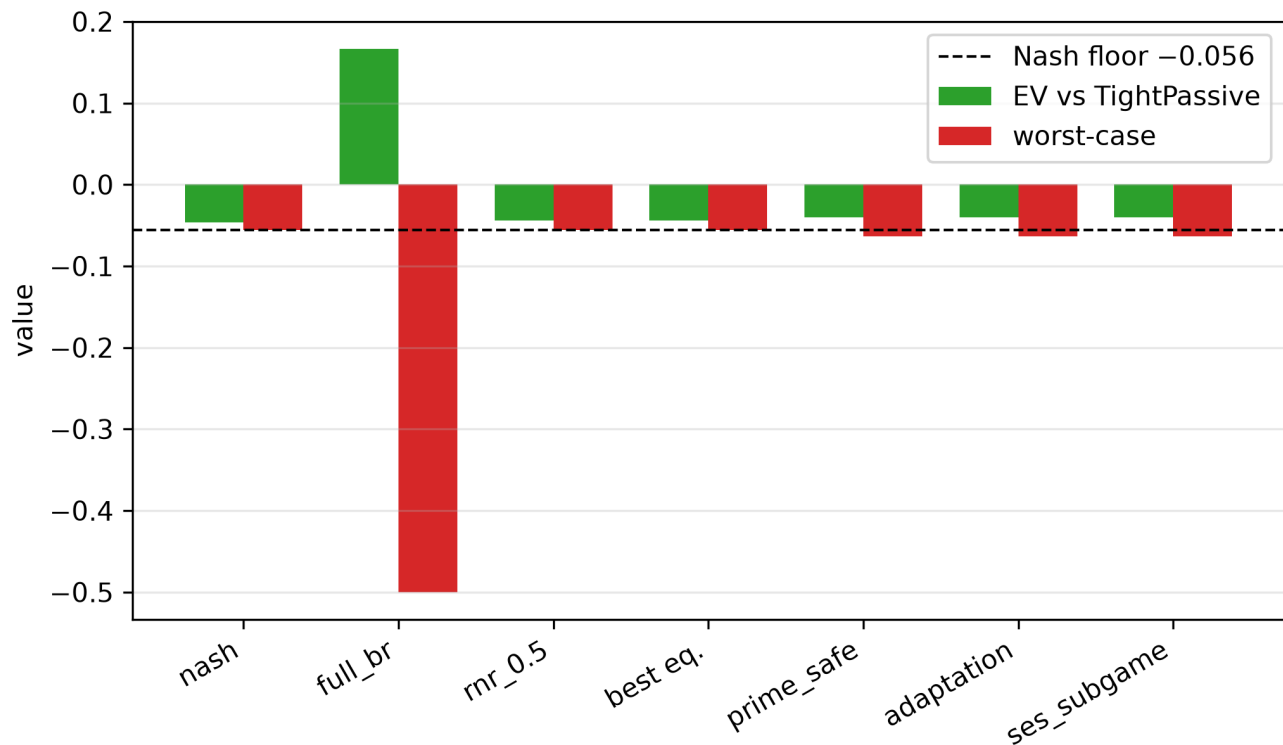


Figure 4: Methods versus the Rock on Kuhn: green is EV against the opponent, red is worst-case value, the dashed line is the Nash floor. Only the full best response’s worst-case (red) plunges to -0.5 , far below the floor; every principled method stays near the floor and earns as much as Nash or slightly more.

Three results carry the chapter.

1. **The full best response is the cautionary tale.** It always wins the most against a fixed model — up to **+0.975** against `AlwaysPass` — but its worst-case collapses to **-0.5** . It is the only method that is unsafe against *every* opponent (`rn timer_0.5` is unsafe against three), because an adversary can always best-respond back to the hole it opens.
2. **Best equilibrium is safe and still profitable — but only slightly.** Against *every* opponent its worst-case stays at the Nash floor (safe within 10^{-3}) **and** it beats Nash’s own EV on every exploitable type — most vividly **+0.222 versus Nash’s +0.146** against `AlwaysPass`, and +0.131 versus +0.118 against the Maniac. This is the result the chapter set out to produce: *you can exploit while provably never dropping below equilibrium value* — but only a little: against the four exploitable types best equilibrium gains 0.002–0.076 per hand over Nash, 1–9 % of what the full best response gains. The gain is small because a per-hand floor at v^* admits only equilibrium strategies (§ 8.3).
3. **A single global p is not a safety setting.** `rn timer_0.5` is safe against the Rock but has *already jumped* to the full best response against the highly exploitable `AlwaysPass` / `AlwaysBet` (identical EV to `full_br`, worst-case -0.33 / -0.17 , unsafe). This is the switching threshold of § 8.5 moving with opponent exploitability — and the concrete reason best equilibrium, which constrains the *value*

rather than a *knob*, is the better primitive.

Prime-safe and adaptation spend a measured ε -budget. They lower the floor from v^* to $v^* - \varepsilon$, and the run *measured* the early-stopped-CFR baseline’s exploitability at $\varepsilon = 0.0074$; their worst-case comes out at $-0.063 = v^* - 0.008$ across every opponent, matching the ε -adjusted floor. Spending that budget, they earn a little more than best equilibrium ($+0.266$ versus $+0.222$ on *AlwaysPass*). They appear “unsafe” in the table only because the flag compares to v^* ; against their *own* floor they are safe by construction. (Prime-safe and adaptation coincide by construction: ε is defined as v^* minus the baseline’s worst case, so $v^* - \varepsilon$ is that worst case, and the implementation uses the same early-stopped CFR strategy as the prime-safe baseline and the adaptation blueprint. Jeary & Turrini’s floor and Ge et al.’s coincide whenever the two are the same strategy; they differ in setting — whole-game exploitation algorithms versus subgame re-solving — not in the floor.)¹

A measurement-resolution note, because it bit me. In the *smoke* run (30 000 CFR iterations) even the Nash policy is flagged “unsafe” — its worst-case is -0.0568 , i.e. 0.0013 below the exact v^* , just over the 10^{-3} tolerance. That is pure CFR approximation error, not a safety failure: at scale (200 000 iterations) the shortfall drops to 3×10^{-4} and Nash is correctly flagged safe. The lesson for the harness is that the safe-flag tolerance must exceed the baseline’s own approximation error, or the baseline trips its own test.

8.7 Real-Time Safety — subgame gadgets (SES)

The guarantee of best equilibrium is *global*: the safety constraint is enforced over the **whole** game tree. That is fine on Kuhn, but in a real game you cannot re-solve the entire tree every time the model updates. The **Safe Exploitation Search** idea (Liu et al. 2022) makes safety a **local** property of a single **subgame**: play the Nash blueprint everywhere, but at a chosen subgame re-optimize the hero’s play to exploit the model — with a **gadget** that bounds how much the local deviation can add to exploitability (the bound grows with the exploitation level and the error of the opponent model). The stricter guarantee used here — never more exploitable than the blueprint — is Ge et al.’s adaptation safety, which their OX-Search achieves for subgame re-solving.

The implementation here enforces that stricter guarantee directly on the treeplex: **pin** every hero sequence *outside* the subgame to its blueprint realization weight (so the hero plays the blueprint everywhere except the subgame), and set the safety floor to the blueprint’s own worst-case value. Because “play the blueprint inside the subgame too” is always feasible and attains exactly that floor, the LP can only do at least as well — the local exploit is safe by construction. The one requirement is that the subgame be **downward-closed** (once you are in it you stay in it), which holds for natural choices like “Leduc round 2” or “after a King flops.”

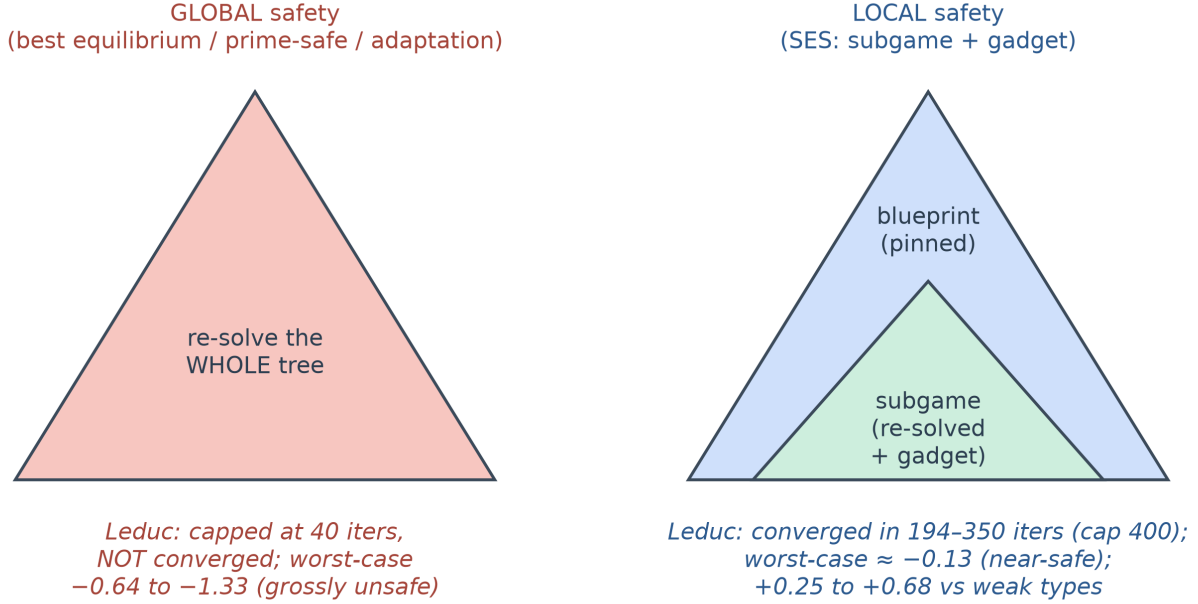


Figure 5: Global versus local safety. Global methods re-solve the whole tree; on Leduc their constraint-generation loop did not converge within the 40-iteration cap and left grossly unsafe strategies. The subgame method pins play outside a chosen subgame to the blueprint and re-solves only that subgame with a gadget — a far smaller problem that, with a cap of 400 iterations, did converge and stayed near-safe.

The importance of “local” is not aesthetic: § 8.8 shows that, on Leduc, the subgame solve converged within its budget and the global solves did not converge within theirs.¹⁰¹¹¹²

8.8 At Scale — Kuhn works; Leduc breaks globally, holds locally

Leduc Hold’em (Section 3.4) is still exactly solvable, but large enough to stress the methods. The Leduc run used an iteration-capped configuration (a 40-iteration budget on the constraint-generation loop, tolerance 10^{-2} , and the subgame set to the King-flop), recording for each cell whether the solve **converged** or hit the cap. Game value $v^* = -0.086$.

¹⁰Liu, M., Wu, C., Liu, Q., Jing, Y., Yang, J., Tang, P. & Zhang, C. (2022). “Safe Opponent-Exploitation Subgame Refinement.” *NeurIPS* — Safe Exploitation Search (SES) with a gadget; the added exploitability is bounded from above (Theorem 4.1).

¹¹Ge, Z. et al. (2024), *op. cit.* — OX-Search: subgame re-solving with a gadget that keeps the refined strategy no more exploitable than the blueprint (adaptation safety, Theorem 4.3), applicable in nested fashion at each newly reached information set; motivated by the “being taught and exploited” problem; tested on Leduc and Flop Hold’em, two-player.

¹²Milec, D., Kovařík, V. & Lisý, V. (2025). “Adapting Beyond the Depth Limit: Counter Strategies in Large Imperfect Information Games.” *AAMAS 2025* (extended abstract), 2675–2677; arXiv:2501.10464 — using opponent-model information *past* the search horizon.

Table 4: Leduc: the subgame method against the global solvers.

Opponent	metric	nash	full_br	ses_subgame	best eq.	prime_safe / adaptation
Rock	EV	+0.201	+0.937	+0.247	+0.624	+0.635
	worst-case	−0.089	−1.633	−0.130	−0.838	−0.744
	converged?	✓	✓	✓ (194 it)	✗ capped	✗ capped
Maniac	EV	+0.438	+2.177	+0.682	+1.806	+1.842
	worst-case	−0.089	−1.100	−0.130	−0.638	−0.657
	converged?	✓	✓	✓ (211 it)	✗ capped	✗ capped
CallingStation	EV	+0.559	+1.464	+0.663	+1.342	+1.363
	worst-case	−0.089	−1.000	−0.130	−0.915	−0.686
	converged?	✓	✓	✓ (350 it)	✗ capped	✗ capped
LoosePassive	EV	+0.449	+1.405	+0.559	+1.306	+1.309
	worst-case	−0.089	−4.200	−0.133	−1.239	−1.334
	converged?	✓	✓	✗ (400 it)	✗ capped	✗ capped

The headline finding is negative and empirical: within a 40-iteration cap, the global solve did not converge even on Leduc. I predicted best equilibrium would be safe on Leduc as it is on Kuhn. Instead the global solvers (best equilibrium `ganzfried`, `prime_safe`, `adaptation`, and `rn timer_0.5`) all hit the 40-iteration cap without converging, leaving worst-case values of **−0.64 to −1.33** — grossly unsafe. The reason, which I confirmed rather than assumed: the cutting-plane loop adds one adversary cut per iteration, and on Leduc’s larger tree the set of relevant pure best responses is large, so 40 cuts nowhere near pin the true worst case; the master LP keeps returning optimistic-but-unsafe strategies. This was flagged as the number-one “likely to break” item *before* the run, and it broke exactly there.

The positive half is the subgame method. SES converged on three of four exploitable opponents (194–350 iterations), because it re-solves only the small King-flop subgame with the rest of the tree pinned — a far smaller LP with a far smaller adversary set. It extracts real value (**+0.25 to +0.68** versus the weak types, beating Nash) at a worst-case of $\approx$ **−0.13**, an order of magnitude closer to safe than the global methods. This is the *global-versus-local safety* distinction of § 8.7 appearing as a measured fact on a game as small as Leduc — the concrete argument for real-time subgame methods and against a naive global solve. The comparison is not at equal budgets, though: the global solvers were stopped after 40 iterations (about 2.5 s per cell), while SES had a cap of 400 and used 194–400 iterations (30–79 s per cell). The result shows that the global loop is slow and unsafe *within this budget*, not that it cannot reach safety with a larger one. Chapter 14 settles the scaling question: its one-shot dual LP, which replaces the loop, solves the full-Leduc problem in 0.02–0.04 s, so the wall was the loop’s, not the global problem’s.

I keep myself honest on SES, though: its residual exploitability ($\approx$ 0.043) still exceeds the 0.01 tolerance,

so it too is flagged unsafe, and on `LoosePassive` it ran the full 400 iterations without converging. That violation is measured against v^* , though. SES’s own floor is the blueprint’s worst case — here the early-stopped CFR baseline (-0.120), not the Nash strategy (-0.089) — and the three converged cells sit 0.010 below it, exactly at the 0.01 tolerance. So SES held its gadget floor to within the tolerance, and the residual is mostly the blueprint’s own exploitability; a rerun with a tighter tolerance would show whether it is *provably* safe.

8.8.1 The teaching attack — and why realized profit is the wrong lens

The last experiment is the deception stress test the whole safety machinery is meant to survive. A deceptive opponent plays the weak Rock bait for 10 000 hands, then switches to a strong Nash “reveal” for 10 000 more; a Chapter 7 model feeds each solver every 500 hands (5 seeds).

Table 5: The teaching attack: realised profit and safety violations per method.

method	mean/hand (all)	mean/hand (after switch)	safety violations / seed
full_br	+0.051	-0.061	40, 40, 40, 40, 40
best eq. (ganzfried)	-0.048	-0.055	0, 0, 0, 0, 0
adaptation	-0.046	-0.055	40, 40, 40, 40, 40
nash	-0.051	-0.061	0, 0, 0, 0, 0

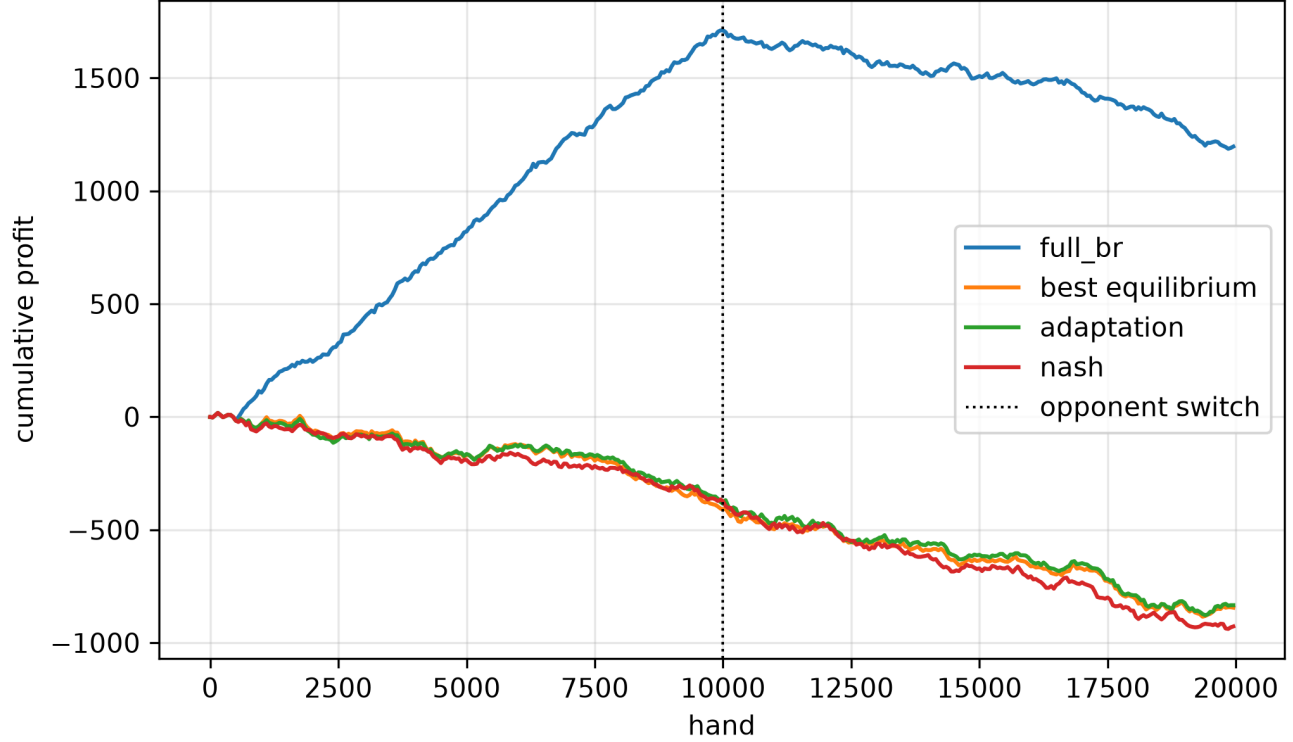


Figure 6: Teaching attack on Kuhn, cumulative profit (the run with seed 0). The full best response (blue) climbs on the bait to about +1700, then only drifts down after the switch — it ends far ahead, because a Nash “revealer” claws back only about the game value per hand. The safe methods refuse the bait and pay the first-player tax throughout.

The naive story — “the teaching attack punishes the greedy exploiter” — **did not hold in realized profit**, and understanding why is instructive. The full best response ends *hugely net-positive* (+0.051/hand overall): it banks a windfall during the bait phase and, because the “reveal” is only a *Nash* opponent, that opponent claws back barely more than the game value per hand, so the windfall is never repaid within 10 000 hands. The signal that *does* separate safe from unsafe is the **exact worst-case / safety-violation count**: the full best response violated the Nash floor at **40 of 40** refits, best equilibrium at **0 of 40**. A strategy’s worst case is realized by an opponent that *best-responds to it* — and a stationary Nash reveal simply is not that adversary. The corrected takeaway, which I now believe is the right design lesson: **measure safety by the worst case, not by realized profit against a benign opponent**; to punish the greedy exploiter *in profit*, the reveal must be an adaptive counter-exploiter, not a fixed Nash. (That *adaptation* shows 40 violations while *nash* shows 0 is not a bug either: adaptation deliberately targets a floor *below* v^* , so it trips the v^* -referenced counter by design.)

8.9 Connections and Forward Pointers

What this chapter establishes. Safe exploitation is one idea — *maximize value against the model subject to a safety floor* — and on a fully solvable game it works exactly as the theory says: best equilibrium is safe against every opponent while earning more than the Nash strategy against every exploitable one, if only slightly, where the naive best response earns more but is ruinously exploitable. Prime-safe and adaptation extend the guarantee to the imperfect baselines any real system has, by spending a *measured* ϵ -budget below the game value. But the chapter’s most valuable result is negative and empirical: on a game as small as Leduc the *global* safe-exploitation solve did not converge within a 40-iteration cap, while the *local* subgame method (cap 400) did — the theory-to-practice gap between global and local safety, measured rather than asserted, if under unequal budgets.

Backward connections. This chapter is the exact complement of the equilibrium and opponent- modeling work before it. Chapter 7’s sensor produces the opponent model that becomes this chapter’s *objective*; Chapter 7’s exact best response becomes this chapter’s *worst-case oracle*; and the sequence-form representation from the CFR and consistency work becomes the *variable* the LP optimizes. The continuous model’s self-inflicted leak against Nash on Leduc (Chapter 7, § 7.7) is the empirical motivation that this chapter’s safety floor is designed to prevent — and, indeed, best equilibrium never loses more than the game value to Nash here.

Forward to the thesis. The Kuhn results show the actuator behaves as the theory predicts (with a perfect opponent model, in an exactly solvable game); the Leduc non-convergence is the concrete argument for the two scalable paths the thesis must choose between — an **exact one-shot dual LP** for the worst-case constraint, or a commitment to **local / subgame** safety (SES, OX-Search) as the real-time mechanism; Chapter 14 took the first path (full Leduc in 0.02–0.04 s). The first addition is Ganzfried and Sandholm’s own gift-risking algorithm (RWYWE, § 8.3), the stronger two-player baseline for Contribution #2; it needs only the existing LP with a floor that moves with the banked gifts, and implementing it is future work. Either way, the safety floor is what turns Chapter 7’s fragile sensor into a deployable adaptive agent.